Lab 2: ekran wyszukiwania

Zaawansowane programowanie na platformę iOS

Na tych zajęciach dodają Państwo drugi ekran aplikacji. Użytkownik wpisuje tytuł albo słowo z opisu filmu i wybiera gatunek. Lista wyników odświeża się sama w trakcie pisania.

Dane nadal są przykładowe, z `Services/SampleMovies.swift`. Wyszukiwanie w TMDB dojdzie na kolejnych zajęciach. Ekran się nie zmieni, podmienią Państwo tylko funkcję, która szuka filmów.

Zanim Państwo zaczną

1. Lab 1 musi być skończony.

Nowe pliki

`MyNextMovie/Models/Search.swift`	`filtrowanie filmów`
`MyNextMovie/ViewModels/SearchViewModel.swift`	`stan ekranu wyszukiwania`
`MyNextMovie/Views/SearchView.swift`	`ekran wyszukiwania`
`MyNextMovieTests/SearchTests.swift`	`testy`

Zmienione pliki:

`MyNextMovie/Services/MovieLoader.swift`	`nowy typ MovieSearch`
`MyNextMovie/Services/SampleMovies.swift`	`nowa funkcja searchSampleMovies`
`MyNextMovie/MyNextMovieApp.swift`	`miejsce na zakładkę Search`

Miejsca do uzupełnienia znajdą Państwo w Xcode: Find Navigator (⌘⇧F), hasło `TODO: Lab 2`.

Jak to działa

Ekran listy z lab 1 dostaje funkcję, która ładuje filmy:

```
typealias MovieLoader = () async throws -> [Movie]
```

Ekran wyszukiwania dostaje funkcję, która szuka filmów po tekście i gatunku:

```
typealias MovieSearch = (_ query: String, _ genreId: Int) async throws ->
[Movie]
```

Teraz ta funkcja to `searchSampleMovies`. Filtruje przykładowe filmy funkcją `filterMovies`. Na kolejnych zajęciach w to miejsce wejdzie zapytanie do TMDB o tym samym typie. ViewModel i widok zostaną bez zmian.

`noGenreId` oznacza „dowolny gatunek”. Pusty tekst oznacza „dowolny tytuł”.

Kiedy użytkownik pisze, każda litera zmienia `viewModel.query`. Widok nie szuka po każdej literze. Czeka 300 ms od ostatniej zmiany i dopiero wtedy woła `viewModel.search()`. Robi to `.task(id:)` w `SearchView`. Gdy `id` się zmienia, SwiftUI przerywa poprzednie zadanie i startuje nowe. Ten kod jest gotowy. Proszę go przeczytać, bo pojawi się na kolejnych zajęciach.

Zadanie 1: logika wyszukiwania

Funkcje w `Models/Search.swift`:

1. `movieContainsText`: czy tekst jest w tytule albo w opisie filmu. Proszę użyć gotowej funkcji `textContains`. Ona ignoruje wielkość liter i polskie znaki.
2. `filterMovies`: filmy, które pasują do tekstu i do gatunku.

Szkielet pętli w `filterMovies`:

```
for movie in movies {
    if warunek_gatunku_nie_pasuje {
        continue
    }
    if warunek_tekstu_nie_pasuje {
        continue
    }
    matchingMovies.append(movie)
}
```

`continue` pomija film i przechodzi do następnego.
Metody w `ViewModels/SearchViewModel.swift`:

1. `search`: szuka filmów i ustawia `state`. Wzorem jest `load` z `MovieListViewModel`. Gdy tekst jest pusty i nie wybrano gatunku, funkcja nie szuka, tylko ustawia `.idle`.
2. `toggleGenre`: pierwsze kliknięcie wybiera gatunek, drugie go odznacza.

Testy do napisania w `MyNextMovieTests/SearchTests.swift`:

1. `FilterMoviesTests/ignoresDiacritics`
2. `FilterMoviesTests/combinesQueryAndGenre`
3. `SearchViewModelTests/passesTrimmedQueryAndGenreToSearch`
4. `SearchViewModelTests/togglingSelectedGenreClearsIt`

Komentarz nad każdym testem mówi, co sprawdzić. Proszę napisać test i usunąć `.disabled(...)`. Test, który woła `await`, musi być `async`.

Po tym zadaniu testy z `SearchTests.swift` przechodzą. Na ekranie jeszcze nic nie widać.

Zadanie 2: ekran

Najpierw zakładka. W `MyNextMovieApp.swift` proszę dodać drugi `Tab` z `SearchView`. Dokładny kod jest w komentarzu `TODO`. Po uruchomieniu aplikacji na dole jest zakładka z lupą. Ekran już działa, ale wyniki to tylko napis z ich liczbą.
Teraz widoki w `Views/SearchView.swift`:

1. `GenreFilter`: wiersz wszystkich gatunków, przewijany w bok. Podobny wiersz jest w lab 1: `GenreRow` w `MovieDetailView.swift`.
2. `genreFilterChip`: `Button`, który woła `toggleGenre`. Wybrany gatunek ma wypełniony chip, reszta jest blada. `GenreChip` ma do tego parametr `isSelected`.
3. `SearchResults`: lista wyników albo komunikat „brak wyników”. Gotowy komunikat to `ContentUnavailableView.search(text:)`.
4. `SearchResultRow`: mały plakat po lewej, tytuł, rok z gatunkiem i ocena po prawej. Kliknięcie otwiera szczegóły filmu, tak jak w siatce z lab 1.

Każdy widok proszę sprawdzać w podglądzie (Canvas, ⌥⌘↩) w `SearchView.swift`. Podgląd używa przykładowych filmów.

Proszę sprawdzić w aplikacji:

1. „matrix” znajduje The Matrix.
2. „wormhole” znajduje Interstellar, bo to słowo jest w opisie.
3. Gatunek Animation bez tekstu pokazuje Spirited Away.
4. „matrix” z gatunkiem Animation pokazuje „No Results”.
5. Drugie kliknięcie w gatunek go odznacza.
6. Kliknięcie w wynik otwiera szczegóły filmu.

Na kolejne zajęcia

Proszę wpisać klucz TMDB do `Config/Secrets.xcconfig`. Instrukcja jest w README, w sekcji Klucze API. Bez klucza aplikacja nie pobierze prawdziwych filmów.

Dokumentacja

- `searchable`: https://developer.apple.com/documentation/swiftui/view/searchable(text:placement:prompt:)
- `task(id:)`: https://developer.apple.com/documentation/swiftui/view/task(id:priority:_:)
- `List`: https://developer.apple.com/documentation/swiftui/list
- `Button`: https://developer.apple.com/documentation/swiftui/button
- `ContentUnavailableView`: https://developer.apple.com/documentation/swiftui/contentunavailableview
- `Tab`: https://developer.apple.com/documentation/swiftui/tab